

Aula 2 — Fases de um Compilador e Especificação de Linguagens

Compiladores I

FACOM - Faculdade de Computação

Bloco 1: Introdução à Compilação

Duração estimada: 70–80 minutos

Revisão da Aula 1

- ▶ Compilador x interpretador;
- ▶ Modelo análise-síntese;
- ▶ As 5 fases do Micro C: léxica, sintática, semântica, código intermediário, Assembly.

Pergunta rápida

Em qual das duas grandes etapas (análise ou síntese) fica a geração de código Assembly?

Objetivos de aprendizagem

- ▶ Identificar e descrever cada fase de um compilador e suas interfaces de entrada/saída;
- ▶ Reconhecer o papel da tabela de símbolos e do tratamento de erros ao longo do processo;
- ▶ Compreender a diferença entre especificação sintática e semântica de uma linguagem.

Sumário da aula

As fases do compilador

Especificação de linguagens

Análise Léxica

Entrada / Saída

Entrada: fluxo de caracteres do código-fonte.

Saída: sequência de **tokens** (unidades léxicas).

Exemplo — Micro C

`int x = 2 + 3;` → tokens: TK_INT, TK_ID(x), TK_ASSIGN,
TK_NUM(2), TK_PLUS, TK_NUM(3), TK_SEMI

Análise Sintática e Análise Semântica

Análise Sintática

Entrada: tokens. Saída: Árvore Sintática Abstrata (ASA), verificando se a sequência de tokens obedece à gramática da linguagem.

Análise Semântica

Entrada: ASA. Saída: ASA anotada + tabela de símbolos, verificando tipos, escopos e regras que a gramática sozinha não captura (ex: variável declarada antes do uso).

Pergunta rápida

`int x = "texto";` é sintaticamente válido? E semanticamente?

Geração de código intermediário e final

Código Intermediário

Representação de nível médio, independente de máquina — no Micro C, o Código de Três Endereços (CTE). Facilita otimizações e portabilidade.

Geração de Código Final

Traduz a representação intermediária para o código-alvo real — no Micro C, Assembly x86-64 (AT&T), com instruções como `mov`, `add`.

Componentes transversais: tabela de símbolos e tratamento de erros

- ▶ **Tabela de símbolos:** estrutura de dados consultada e atualizada por várias fases (nomes, tipos, escopos, endereços);
- ▶ **Tratamento de erros:** cada fase pode detectar um tipo de erro diferente — léxico (caractere inválido), sintático (token inesperado), semântico (tipo incompatível);
- ▶ Um bom compilador reporta erros de forma clara, sem interromper a análise no primeiro erro encontrado, quando possível.

Sumário da aula

As fases do compilador

Especificação de linguagens

Sintaxe x Semântica

Sintaxe

Define a **forma** válida dos programas (o "o que pode ser escrito").
É especificada por gramáticas formais.

Semântica

Define o **significado** dos programas sintaticamente válidos. É bem mais difícil de formalizar; frequentemente descrita em linguagem natural ou por regras de tipos.

Exemplo —

$x = x / 0$; é sintaticamente correto, mas tem um problema semântico (em tempo de execução).

Desafio em sala

Desafio em sala

Supondo que as variáveis a , b e c são do tipo inteiro, para o comando `if (a > b) c = a; else c = b;` do Micro C, modifique, minimamente, o trecho de código de forma a incluir: (1) um exemplo de erro léxico, (2) um exemplo de erro sintático e (3) um exemplo de erro semântico que poderiam ocorrer nessa linha.

Síntese da aula

- ▶ Cada fase do compilador tem entrada/saída bem definidas e um tipo de erro característico;
- ▶ A tabela de símbolos e o tratamento de erros atravessam várias fases;
- ▶ Sintaxe define forma; semântica define significado.

Próxima aula: gramáticas livres de contexto e gramáticas ambíguas.

Teste rápido — Aula 2

Questões teóricas

1. Diferencie erro léxico, erro sintático e erro semântico, dando um exemplo de cada.
2. Qual é a entrada e a saída da análise semântica?

Questão prática

3. Para `float y = 10 / "a";`, aponte em qual fase do compilador o erro seria detectado e por quê.